

Recurrent Neural Networks

RNNs for NLP Tasks

Introduction to Deep Learning · Class 29

Outline

The Recurrent Language Model

Labelling and Classification

Stacking and Direction

The RNN as a Probabilistic Model

Putting It Together

The Recurrent Language Model

The cell, put to work

- Class 27 built the encoder–decoder on a cell g ; Class 28 opened it. Today the cell is fixed and the question is **what to connect to it**
- Four output structures: a token at every step (**language modelling**), a label at every step (**sequence labelling**), one label for the whole sequence (**classification**), a new sequence (the **encoder–decoder**, Class 27)
- Two ways to combine cells: **stack** them, and run one **in each direction**
- Closing: the RNN as a probabilistic model, and its tree-shaped cousin

Jurafsky & Martin (2026), §14.2–14.4, §14.6; Goodfellow et al. (2016), §10.2.3, §10.5–10.6.

Notation for today

$\mathbf{x}_t \in \{0, 1\}^{ V }$	one-hot token, index w_t ; $\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$ its embedding, $\mathbf{E} \in \mathbb{R}^{d \times V }$
d	the model dimension, shared by embeddings and states
$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$	the cell of Classes 27–28 (simple, or an LSTM with $\mathbf{c}_t$ inside)
$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$	output distribution, $\mathbf{V} \in \mathbb{R}^{ V \times d}$; $\hat{y}_t[k]$ its k -th entry
$\mathbf{h}_t^f, \mathbf{h}_t^b, [\cdot; \cdot]$	forward and backward states; concatenation
$L_{\text{CE}}, \text{PPL}$	cross-entropy loss; perplexity
$y^{(t)}, h^{(t)}, \tau$	Goodfellow's superscripts and length, in the graphical-model slides only

Two conventions, one line

Jurafsky: $\mathbf{W}$ input, $\mathbf{U}$ recurrence, $\mathbf{V}$ output, time as a subscript. Goodfellow swaps $\mathbf{U}$ and $\mathbf{W}$ and writes $y^{(t)}$. This deck keeps Jurafsky's letters, as Classes 27–28 did.

Jurafsky & Martin (2026), §14.2.1; Goodfellow et al. (2016), §10.2.

The recurrent language model, recalled

Definition 1 (RNN language model)

At each step t ,

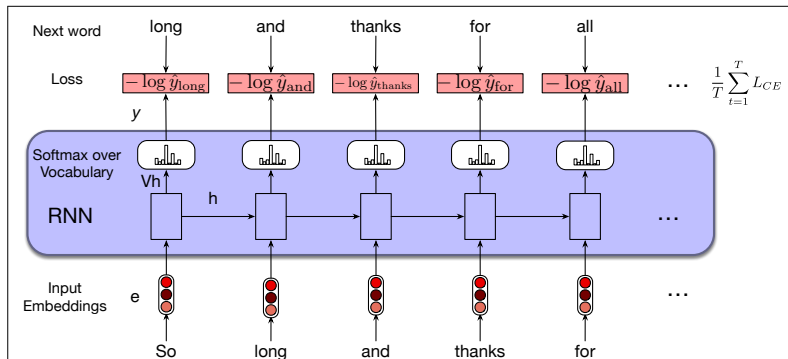
$$\mathbf{e}_t = \mathbf{E}x_t, \quad \mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t),$$

and $P(w_{t+1} = k \mid w_{1:t}) = \hat{y}_t[k]$. The probability of a sequence is the product of its next-token probabilities, $P(w_{1:n}) = \prod_{i=1}^n \hat{y}_i[w_i]$.

- Against the n -gram model, the context is not $n - 1$ tokens but **everything so far**, through $\mathbf{h}_{t-1}$; against the feedforward model, not a fixed window
- Shapes: $\mathbf{E}$ is $d \times |V|$, $\mathbf{U}, \mathbf{W}$ are $d \times d$, $\mathbf{V}$ is $|V| \times d$: the vocabulary enters only at the two ends

Jurafsky & Martin (2026), Eqs. 14.4–14.9, Fig. 14.5; Mikolov et al. (2010).

Training by self-supervision



Training an RNN as a language model. At every position the model reads the true word, predicts the next, and pays the cross-entropy of the true next word, $L_{CE} = -\log \hat{y}_t[w_{t+1}]$; the loss is the average over the sequence. No labels are added: the text is its own supervision. Teacher forcing: the input at $t + 1$ is the true w_{t+1} , not the prediction.

Jurafsky & Martin (2026), Fig. 14.6, §14.2.2, Eqs. 14.10–14.11.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals exp of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- **Proof.** By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- Reading: the **effective number of choices** per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

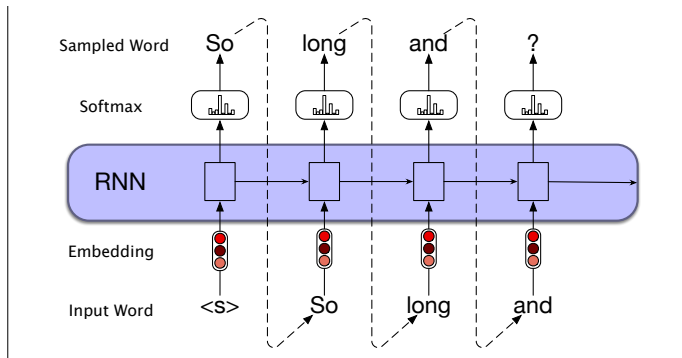
For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals exp of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- Proof. By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- **Reading:** the effective number of choices per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

Generation, drawn



Autoregressive generation with an RNN language model: the sampled word at each step becomes the input at the next, starting from $\langle s \rangle$ and stopping at $\langle /s \rangle$ or a length limit. The RNN's hidden layers and recurrent connections are inside the block. Jurafsky & Martin (2026), Fig. 14.9, §14.3.3.

Generation: sampling, one token at a time

- **Autoregressive generation:** feed $\langle s \rangle$, sample a word from $\hat{y}_1$, feed its embedding back, sample again; stop at $\langle /s \rangle$ or a length limit. The text is a sample from $P(w_{1:n})$, not its mode
- **Temperature:** $\hat{y} = \text{softmax}(u/\tau)$; $\tau < 1$ sharpens towards the likely words, $\tau > 1$ flattens, $\tau \rightarrow 0$ is greedy decoding
- The same loop is Class 27's decoder, started from a context vector instead of $\langle s \rangle$: the priming makes it translation, summarisation or an answer

Class 27's greedy and beam decoding pick likely sequences; sampling picks typical ones. A language model is a distribution, and both are ways of reading it.

Jurafsky & Martin (2026), §14.3.3, Fig. 14.9, §7.6.2–7.6.3, Eq. 7.51; Shannon (1951).

Weight tying

Definition 3 (Tied language model)

The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a *tied* model uses one matrix for both:

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

Proposition 4 (What tying saves)

The embedding and output layers cost $|V| d$ instead of $2|V| d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- **Proof.** $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

Weight tying

Definition 3 (Tied language model)

*The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a **tied** model uses one matrix for both:*

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

Proposition 4 (What tying saves)

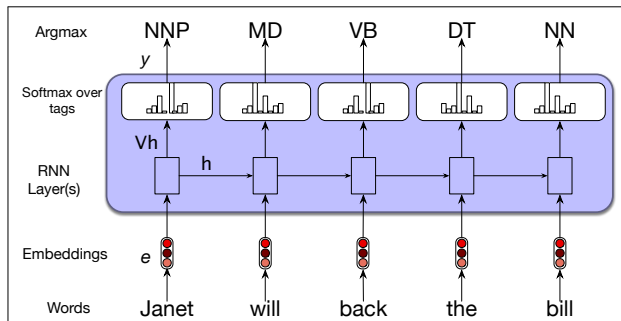
The embedding and output layers cost $|V| d$ instead of $2|V| d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- Proof. $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- **The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.**

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

Labelling and Classification

Sequence labelling: a label at every step



Part-of-speech tagging as sequence labelling with a simple RNN: pre-trained word embeddings in, an RNN, and a softmax over the tag set at every step; the tag is the argmax. The language model's output layer with the tag set in place of the vocabulary, the cross-entropy summed over the steps, and a local decision at each.

Jurafsky & Martin (2026), Fig. 14.7, §14.3.1.

Sequence classification: one label per sequence

Definition 5 (Read-outs for classification)

Run the RNN over $x_1 \dots x_n$ and pass one vector to a feedforward classifier with a softmax over the classes; three choices of the vector:

$$\mathbf{h}_n \text{ (the last state),} \quad \mathbf{h}_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i, \quad \mathbf{h}_{\text{max}} = \max_i \mathbf{h}_i \text{ (elementwise).}$$

- No loss at earlier positions: the one error signal goes back through the classifier, then the RNN — **end-to-end training**. $\mathbf{h}_n$ must carry the whole text; pooling reads every state

Jurafsky & Martin (2026), §14.3.2, Eq. 14.15, Fig. 14.8.

Why the read-out matters

Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

- **Proof.** Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{I}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If the evidence is at the start of a long text, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

Why the read-out matters

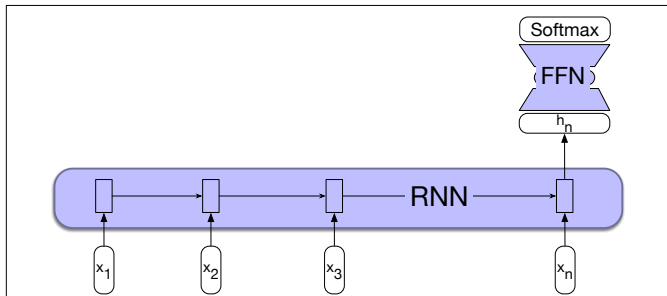
Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

- Proof. Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{I}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If the evidence is at the start of a long text, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

Sequence classification, drawn

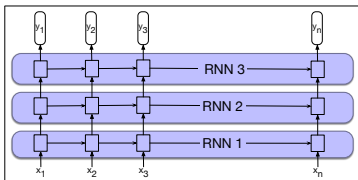


Sequence classification with a simple RNN and a feedforward network: the final hidden state h_n is the input to the classifier, and the only loss is the classification's, back-propagated through the whole network.

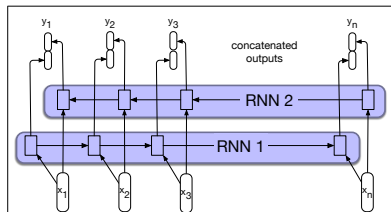
Jurafsky & Martin (2026), Fig. 14.8, §14.3.2.

Stacking and Direction

RNNs as modules



(a) stacked recurrent networks: the output of a lower level serves as the input to higher levels, and the last network's output is the final output.
Jurafsky & Martin (2026), Fig. 14.10.



(b) a bidirectional RNN: separate models in the forward and backward directions, their outputs concatenated to form the bidirectional state at each step.
Jurafsky & Martin (2026), Fig. 14.11.

- Unrolled, an RNN is a feedforward graph with vectors in and out, so it composes like any other layer

Jurafsky & Martin (2026), §14.4.

Stacking: depth per time step

Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- **Proof.** Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. □
- Goodfellow's three ways to make an RNN deep: (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking: depth per time step

Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- Proof. Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. $\square$
- **Goodfellow's three ways to make an RNN deep:** (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking: depth per time step

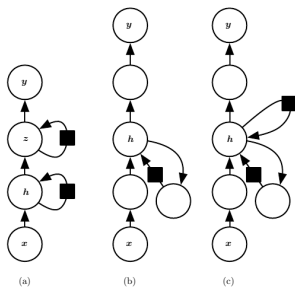
Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- Proof. Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. $\square$
- Goodfellow's three ways to make an RNN deep: (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- **Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.**

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Three ways to make an RNN deep



- (a) the hidden state broken into groups organised hierarchically — the stacked network of Fig. 14.10
- (b) deeper computation, an MLP, in the input-to-hidden, hidden-to-hidden and hidden-to-output parts; this lengthens the shortest path between time steps
- (c) skip connections in the hidden-to-hidden path, which shorten it again

A recurrent network can be made deep in many ways.

Goodfellow et al. (2016), §10.5, Fig. 10.13, after Pascanu et al. (2014).

Bidirectionality: the right context

Definition 8 (Bidirectional RNN, as used)

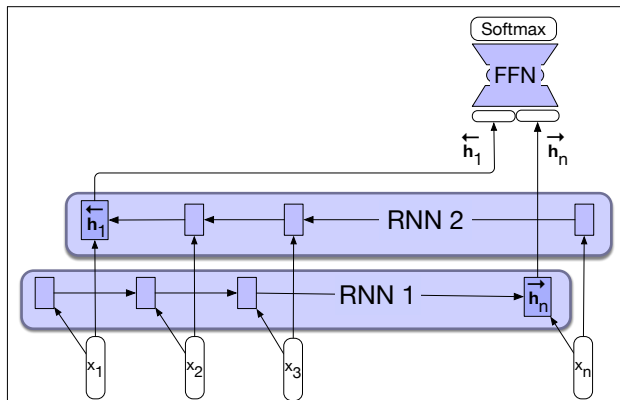
Two independent RNNs, concatenated at each step:

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(x_1, \dots, x_t), \quad \mathbf{h}_t^b = \text{RNN}_{\text{backward}}(x_t, \dots, x_n), \quad \mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b].$$

- $\mathbf{h}_t^f$ is the ordinary state, everything to the left; $\mathbf{h}_t^b$ the state of a network run on the reversed input, everything to the right
- **Labelling**: the decision at t sees both sides. **Classification**: concatenate the two final states, so the beginning counts as much as the end. Needs the whole input, so not for generation

Jurafsky & Martin (2026), §14.4.2, Eqs. 14.16–14.18, Figs. 14.11–14.12; Schuster & Paliwal (1997); Class 26.

Bidirectional classification, drawn



A bidirectional RNN for sequence classification: the final hidden units from the forward and backward passes are combined (here by concatenation) and passed to the feedforward classifier, so the representation is not dominated by the end of the sentence. Jurafsky & Martin (2026), Fig. 14.12, §14.4.2.

The RNN as a Probabilistic Model

The RNN as a directed graphical model

Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

- **Proof.** The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. $\square$
- The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The RNN as a directed graphical model

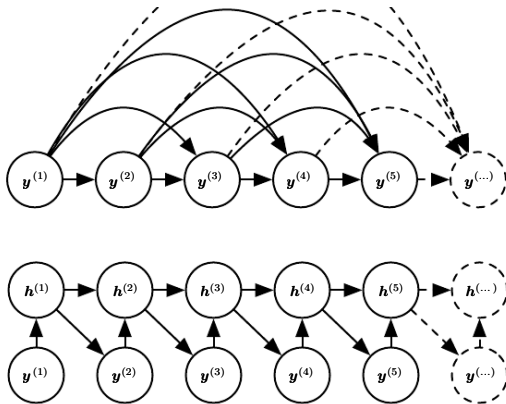
Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

- Proof. The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. □
- **The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.**

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The two graphs, drawn



Top: the fully connected graphical model over $y^{(1)}, y^{(2)}, \dots$: every past observation may influence the conditional distribution of a later one, and parametrising this graph directly is inefficient. Bottom: with the state variable $h^{(t)}$ in the graph, even though it is a deterministic function of its inputs, every stage has the same structure and can share the same parameters.

Goodfellow et al. (2016), Figs. 10.7–10.8, §10.2.3.

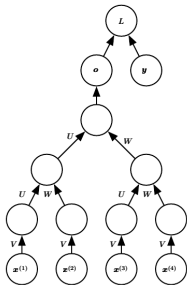
Deciding when to stop

- A model over sequences must also decide their **length**. Three mechanisms, all trained from data:
- An **end symbol** $\langle /s \rangle$ in the vocabulary, inserted after the last token of every training sequence; sampling stops when it is drawn. This is Class 27's decoder and today's language model
- A **Bernoulli output** at every step, a sigmoid trained by cross-entropy to say “continue” or “halt”; works for any output type, real-valued sequences included
- **Predict τ first**, then sample τ steps, feeding τ or $\tau - t$ back as an extra input so the model knows how close the end is:

$$P(x^{(1)}, \dots, x^{(\tau)}) = P(\tau) P(x^{(1)}, \dots, x^{(\tau)} \mid \tau)$$

Goodfellow et al. (2016), §10.2.3, Eq. 10.34; Schmidhuber (2012).

The tree-shaped cousin: recursive networks



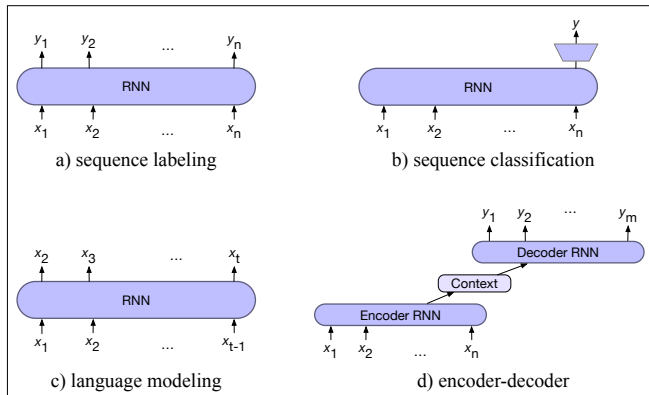
A recursive network: the computational graph generalises the recurrent chain to a tree, with a fixed set of parameters U , V , W and a fixed-size output o for a variable-size sequence. Goodfellow et al. (2016), Fig. 10.14.

- The same idea as the RNN — one function, one parameter set, applied repeatedly — on a **tree** instead of a chain; a variable-length sequence still maps to one vector
- Depth: the number of nonlinear compositions from a leaf to the root is $O(\log \tau)$ for a balanced tree, against τ for the chain, which helps with long dependencies
- The open question is the tree: fixed and balanced, given by a parser (the sentence's parse tree, Socher et al.), or inferred by the learner

Goodfellow et al. (2016), §10.6, Fig. 10.14; Pollack (1990); Socher et al. (2011, 2013).

Putting It Together

Four architectures



Four architectures for NLP tasks. In sequence labelling each input token x_i maps to an output token y_i ; in sequence classification the entire input maps to one class; in language modelling the next token is output conditioned on the previous ones; in the encoder-decoder one RNN maps the input to a context and a second maps the context to an output sequence.

Jurafsky & Martin (2026), Fig. 14.15, §14.6.

The block, in one table

	output	loss	where the gradient enters
language model	$\hat{y}_t$ over V , every step	$-\log \hat{y}_t[w_{t+1}]$, averaged	every step
sequence labelling	$\hat{y}_t$ over tags, every step	cross-entropy, every step	every step
classification	one softmax	one cross-entropy	$\mathbf{h}_n$, or every $\mathbf{h}_t$ if pooled
encoder-decoder (27)	$\hat{y}_t$ over V , decoder steps	teacher-forced cross-entropy	decoder steps, back through $\mathbf{c}$
stacked	top layer's outputs	as above	through L cells per step
bidirectional	$[\mathbf{h}_t^f; \mathbf{h}_t^b]$	as above	from both directions

One cell, three matrices, and the choice of what to read out and where to pay the loss: that choice is the architecture.

Takeaways

1. A language model is an RNN whose output is the next token; it trains on raw text by self-supervision, and its loss is the log of its perplexity (Definition 1, Proposition 2)
2. Tying the output matrix to the embeddings removes $|V|d$ parameters (Definition 3, Proposition 4); sampling with a temperature reads the model as a distribution
3. Labelling emits a distribution at every step; classification reads out one vector, and pooling gives the early tokens a direct path to the loss (Definition 5, Proposition 6)
4. Stacking buys depth per step at the same parameter count (Proposition 7); bidirectionality buys the right context (Definition 8)
5. The RNN is the complete graphical model over the sequence with $O(1)$ parameters in its length (Proposition 9); the recursive net is the same idea on a tree

Classes 30–31: the recurrence goes, attention stays — the transformer.

References i

Questions?